Design a Game Control Panel

Year 5/6 STEAM | Design and Technologies with Digital Technologies | 2 x 45 minute lessons

Overview

Students design, build and test a control panel that lets a brand new player use a micro:bit and Jacdac game without being told how. The focus is the **user**, not decoration. Because the task includes repeated user testing and optional **level up** challenges, there is no natural finish line for fast finishers.

Design brief: Design a control panel so that someone who has never played the game (a Prep student, a parent at open day) can work it out without being told.

Student choice: each pair designs a panel for either the new **Reaction Game** or their own **Button Smash** game from last week. Same brief, same criteria.

Materials

- Jacdac kit per pair: micro:bit, 2 Jacdac buttons, 8 pixel LED ring, cables
- iPads with the micro:bit app
- Shoebox lids or A5 squares of corrugated card (one per pair, keeps panels small and stackable)
- Blu Tack and Velcro dots so components come off and go back in the kits
- Markers, coloured dot stickers, blank labels
- STEAM journals
- Level up challenge cards (see below), printed or displayed

Lesson 1: Code, Play and Plan

Learning Intention:

To find the problems players have with our game and plan a control panel that solves them.

Success Criteria:

I can:

- connect and run my game code.
- identify problems a new player has when using the game.
- sketch a control panel layout and explain why I put each part where I did.

Time	Activity	Teacher notes
0 to 5 min	**Hook.** Hold up a Jacdac kit and explain the Reaction Game: press one button to start, wait for green, hit the other button as fast as you can. Introduce the design brief and the choice of game.	Keep it brief. Students discover the design problems themselves once they play.
5 to 17 min	**Get the game running.** Reaction Game pairs build or load the code. Button Smash pairs reopen their project and add a second Jacdac button if they want one.	Students press each button while watching the simulator to find out which is button1 and which is button2.
17 to 24 min	**Play and notice.** Pairs play, then ask: *Would a Prep student know how to play this?* They list in their journal every moment they were confused or made a mistake.	Point out that the two buttons look identical. That confusion is the design problem. Prompt: Where did your eyes go? Could you read the score?
24 to 29 min	**Build criteria together.** Pairs share problems. Turn them into 3 or 4 class design criteria and record them on the board.	See example criteria below. Keep them about the player, not about looks.
29 to 40 min	**Sketch two layouts** in the journal, annotated with why each part sits where it does.	Fast finishers pick up a level up card and think about how it would change their layout.
40 to 45 min	**Choose and justify.** Circle one layout and write one sentence linking it to a criterion. Pack kits away.	Collect journals or check one justification per pair as an exit check.

Example design criteria

- A new player can tell which button starts the game.
- The two buttons can't be confused with each other.
- The player can see the LED ring and reach the button at the same time.
- The score on the micro:bit is easy to read.

Lesson 2: Build, Test and Improve

Learning Intention:

To build and test a control panel so that a new player can use our game without being told how.

Success Criteria:

I can:

- build my control panel from my plan.
- test my panel with two different users and record what they found difficult.
- make an improvement and explain why I made it.

Time	Activity	Teacher notes
0 to 5 min	**Revisit.** Recap the class criteria and model the silent test protocol.	Rehearse the no talking rule. It is harder than students expect.
5 to 22 min	**Build** the panel on a shoebox lid or A5 card, components attached with Blu Tack or Velcro.	Redirect any pair spending time on decoration back to the criteria: Which problem does that solve?
22 to 28 min	**Silent test, round 1.** Swap panels with another pair.	Observers tally each hesitation or mistake in their journal.
28 to 34 min	**Silent test, round 2** with a different pair.	A second tester shows whether a problem was a one-off or a real design flaw.
34 to 41 min	**Improve.** Make at least one change based on the tests and annotate it in the journal.	Fast finishers take on a level up card and update the panel to match.
41 to 45 min	**Record and pack up.** Photograph the panel for the journal. Components back in kits, panels stacked flat in the class tray.	Nothing goes home and nothing needs storing except flat panels.

Silent test protocol

- The tester gets no instructions. The makers say nothing, even if the tester gets stuck.
- Makers record in their journal every time the tester hesitates, asks, or presses the wrong thing.
- After the test the tester finishes one sentence: *I was confused when...*

Level Up Challenges

These keep fast finishers working and push them back into the code. Each one changes what the panel needs to communicate, so students must update their design too.

Challenge	Code change	Panel change
Too early	Pressing before green turns the ring red. Add an **else** to the reaction button's **if** block.	How will a new player know the rule? Add a warning or instruction.
Best time	Make a **bestTime** variable and update it when a new time is faster.	Where does the best time appear, and how does the player see it?
Speed ring	Light more pixels for faster times using **setPixelColor**.	Add a key so the player understands what the ring means.
Two player	Add a third button. The first player to press after green wins.	Redesign for fairness: equal distance, clear player sides.

Reaction Game Code

Paste into the JavaScript view in MakeCode, then switch to Blocks. Add the Jacdac Button and Jacdac LED extensions first.

```
let startTime = 0
let reactionTime = 0
let canPress = false

// Press the start button to begin
modules.button2.onEvent(jacdac.ButtonEvent.Down, function () {
    basic.clearScreen()
    modules.led1.setAll(0x000000)
    basic.pause(randint(1000, 5000))
    modules.led1.setAll(0x00ff00)
    startTime = input.runningTime()
    canPress = true
})

// Press the reaction button when the ring turns green
modules.button1.onEvent(jacdac.ButtonEvent.Down, function () {
    if (canPress) {
        canPress = false
        reactionTime = input.runningTime() - startTime
        basic.showNumber(reactionTime)
    }
})
```